DataCollection1-S26

February 16, 2026

1 Team Name: [B Team]

2 Authors: [Madelyne Dusbabek, Gregory Lowman, Jay Hall.]

3 Cell Number: [R22]

4 J-V Apparatus Number: [JV1]

5 EQE Apparatus Number: [EQE1]

6 Reminder: For consistency across team notebooks, please include Markdown cells and inline comments to clarify key steps.

You can also find some additional boilerplate code for creating subplots here.

6.1 J-V Analysis

Now that you have two weeks of data, you will begin to make plots that show how the J_{sc} and PCE change with time. The measurements you took last week prior to stressing will be at time equal to zero hours and the measuremens this week will be after approximately 168 hours of stressing.

1) Calculate the J_{sc} for each of your pixels using the **forward scans**. Remember you can copy and paste code from previous notebooks and use AI tools. Consider using a loop in your python code more be more efficient.

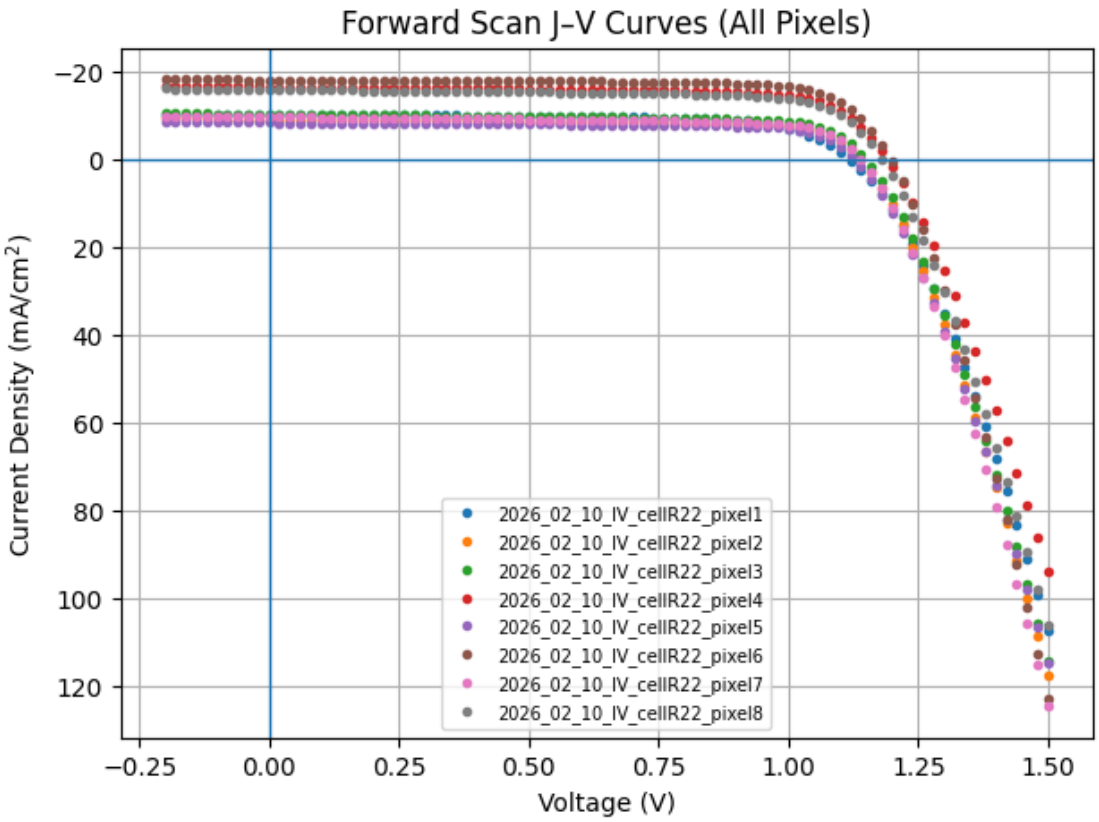
7 JV graph after 168 hrs

[18]:
```
#Mount to google drive to use shared files

from google.colab import drive
drive.mount("/content/drive", force_remount=True)

#Import libraries to do math and build the plots neccesary

import pandas as pd
import numpy as np
from matplotlib import pyplot as plt
```

```
import os

#Constant for the area of a solar cell

AREA = 0.14  # cm^2

#Select a folder to pull CSV files from

folder = "/content/drive/MyDrive/Section 104 Tuesday 2:15 PM Team B/Week3/
 ↪JV_Data"

#Create an array of files

files = sorted(os.listdir(folder))
jv_data = {}

#Loop through each file in the folder / array

for f in files:
    #Create appropriate file pathways

    df = pd.read_csv(folder + "/" + f)

    jv_columns = {
        "Voltage (V)": df["Voltage (V)"],
        "Current Density (mA/cm^2)": df["Forward_mean (mA)"] / AREA
    }

    #Name each data set without the .csv
    name = f.split("/")[-1].replace(".csv", "")
    jv_data[name] = pd.DataFrame(jv_columns)

#Create a plot for the data

plt.figure(figsize=(7,5))

#Plot all of the datasets (loop), not just one

for name, jv in jv_data.items():
    plt.plot(jv["Voltage (V)"], jv["Current Density (mA/cm^2)"], ".",␣
 ↪label=name)

#Plot attributes

plt.axhline(0, linewidth=1)
plt.axvline(0, linewidth=1)
plt.xlabel("Voltage (V)")
```

```
plt.ylabel("Current Density (mA/cm$^2$)")
plt.title("Forward Scan J-V Curves (All Pixels)")
plt.legend(fontsize=7)
plt.grid(True)

# I inverted the axis bc that's how they showed this data in lecture
plt.gca().invert_yaxis()
plt.show()
```

```
Mounted at /content/drive
```



2) Calculate the Power Conversion Efficiency (PCE) for each of your pixels using the **forward scans**.

a) Remember you can copy and paste code from previous notebooks and use AI tools. Consider using a loop in your python code more be more efficient.

b) The light power hitting the pixel has been set to $P_{in} = 99.8\ mW/cm^2$.

8 PCE, Voc, Jsc, Vpmax, Jpmax calculations

[19]:
```
# Power in
Pin = 99.8 # mW/cm^2

results = [] # Create table

# Loop through files created in JV section above
for file in files:
  file_path = os.path.join(folder, file) # Combines foler and files strings
  df = pd.read_csv(file_path) # Create dataframe of file_path
  V = df['Voltage (V)'] # Creates voltage vector
  J = df['Forward_mean (mA)']/0.14 # Creates current density vector

  Jsc = df.loc[np.isclose(df['Voltage (V)'], 0), "Forward_mean (mA)"].iloc[0]/0.
 ↪14 #Find short circuit current when voltage is 0
  idx_voc = df['Forward_mean (mA)'].abs().idxmin() # Locates index where␣
 ↪current is lowest
  Voc = V.loc[idx_voc] # Assigns voltage in voltage vector at idx_voc to Voc

  P = J * V # Creates power vector as a product of current density and voltage␣
 ↪vector
  Pmax = P.min() # Maximum power point is most negative value (see JV curve␣
 ↪overlayed w/ power)
  idx_max = P.idxmin() # Sets index location of lowest value
  Vpmax = V.loc[idx_max] # Voltage at mpp is at same index as mpp
  Jpmax = J.loc[idx_max] # Current density max is at same index as mpp

  PCE = -((Jpmax * Vpmax)/Pin)*100 # Calculate PCE

  # Edit pixel label
  name_without_csv = os.path.splitext(file)[0]
  pixel_label = name_without_csv.split('_')[-1].replace('pixel','')
  pixel_label = int(pixel_label)

  # Append results list
  results.append({
      "Pixel": pixel_label,
      "Voc (V)": round(Voc, 4),
      "Jsc (mA/cm²)": round(Jsc, 4),
      "Vpmax (V)": round(Vpmax, 4),
      "Jpmax (mA/cm²)": round(Jpmax, 4),
      "Pmax (mW/cm²)": round(Pmax, 4),
      "PCE (%)": round(PCE, 4)
  })

# Create table
```

```
table = pd.DataFrame(results)

# Display table
display(table)
```

```
   Pixel  Voc (V)  Jsc (mA/cm²)  Vpmax (V)  Jpmax (mA/cm²)  Pmax (mW/cm²)  \
0      1     1.12      -10.1636       0.92         -8.7653        -8.0641
1      2     1.14       -9.0380       0.98         -7.7236        -7.5692
2      3     1.14      -10.2417       1.00         -8.6042        -8.6042
3      4     1.20      -16.4786       1.00        -14.6104       -14.6104
4      5     1.12       -8.3315       0.98         -7.1168        -6.9744
5      6     1.20      -18.0901       1.02        -16.2689       -16.5943
6      7     1.14       -9.6249       0.98         -8.0859        -7.9241
7      8     1.18      -15.8427       1.00        -13.7969       -13.7969

   PCE (%)
0   8.0802
1   7.5843
2   8.6215
3  14.6397
4   6.9884
5  16.6276
6   7.9400
7  13.8246
```

3) Create a csv file with J_{sc} and PCE values you just calcualted for this week's measurements. This will allow you to easily reference previous weeks' data and plot data from multiple weeks at once. **Be sure to download the csv file you create and add it to your team's Google Drive folder.**

We have written the code for you to do this below. You just have to make sure the variable and file names are the ones you used.

9 New .csv with Jsc and PCE

[21]:
```
# Create new table with only desired data from 'table' above
export = table[["Pixel", "Jsc (mA/cm²)", "PCE (%)"]]

# display(export)

output_path = "/content/drive/MyDrive/Section 104 Tuesday 2:15 PM Team B/Week3/
 ↪2026_02_10_jsc_pce_cellR22.csv"
export.to_csv(output_path, index=False)

print("Saved to:", output_path)
```

```
Saved to: /content/drive/MyDrive/Section 104 Tuesday 2:15 PM Team
B/Week3/2026_02_10_jsc_pce_cellR22.csv
```

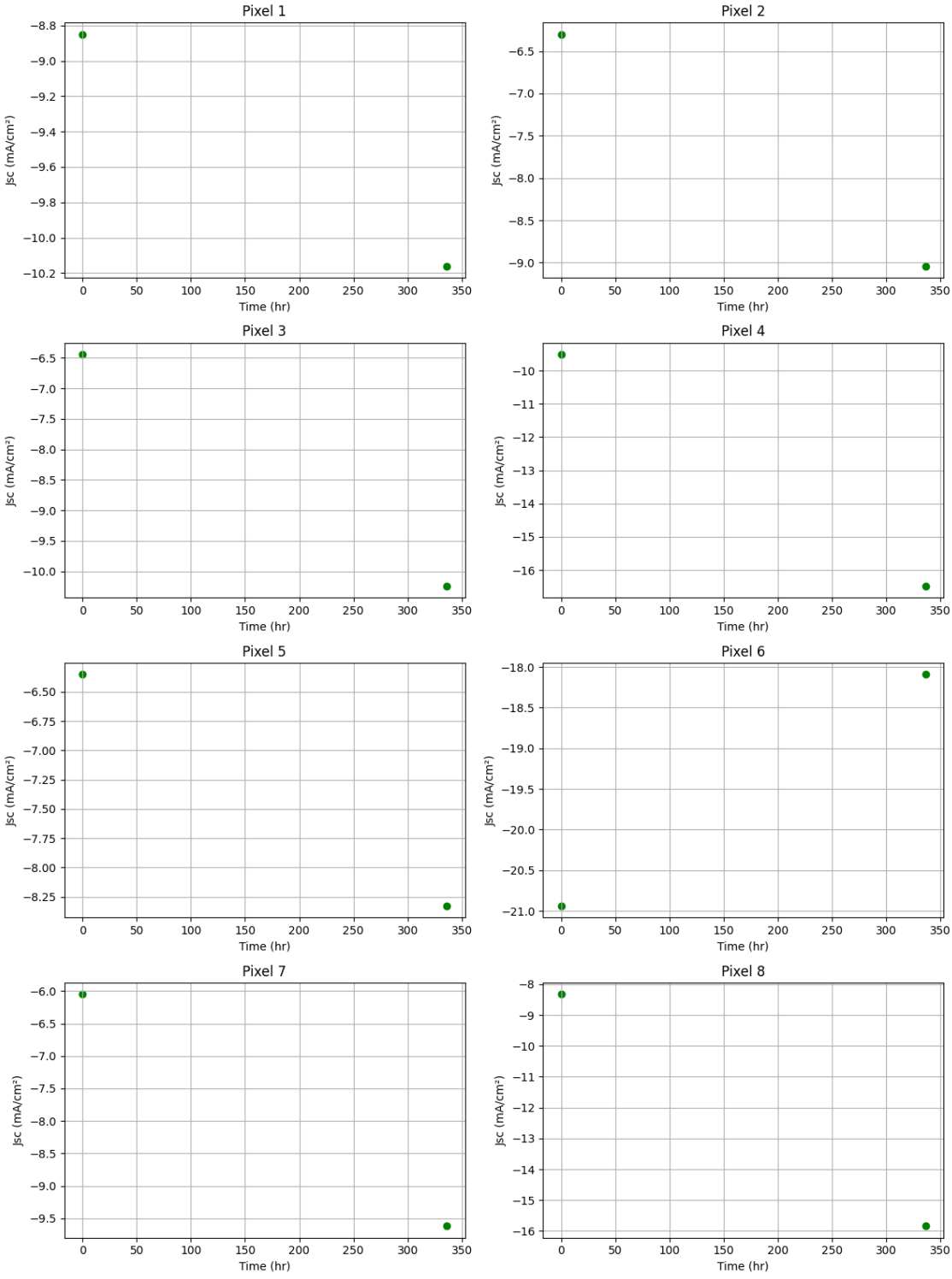
4) Copy and paste the code in the above cell into your Colab notebook from **the first time you took J-V data** to create a csv file of those J_{sc} and PCE values as well. Make sure the varible and file names are correct. **You will need to change at least the date in csv file name.**

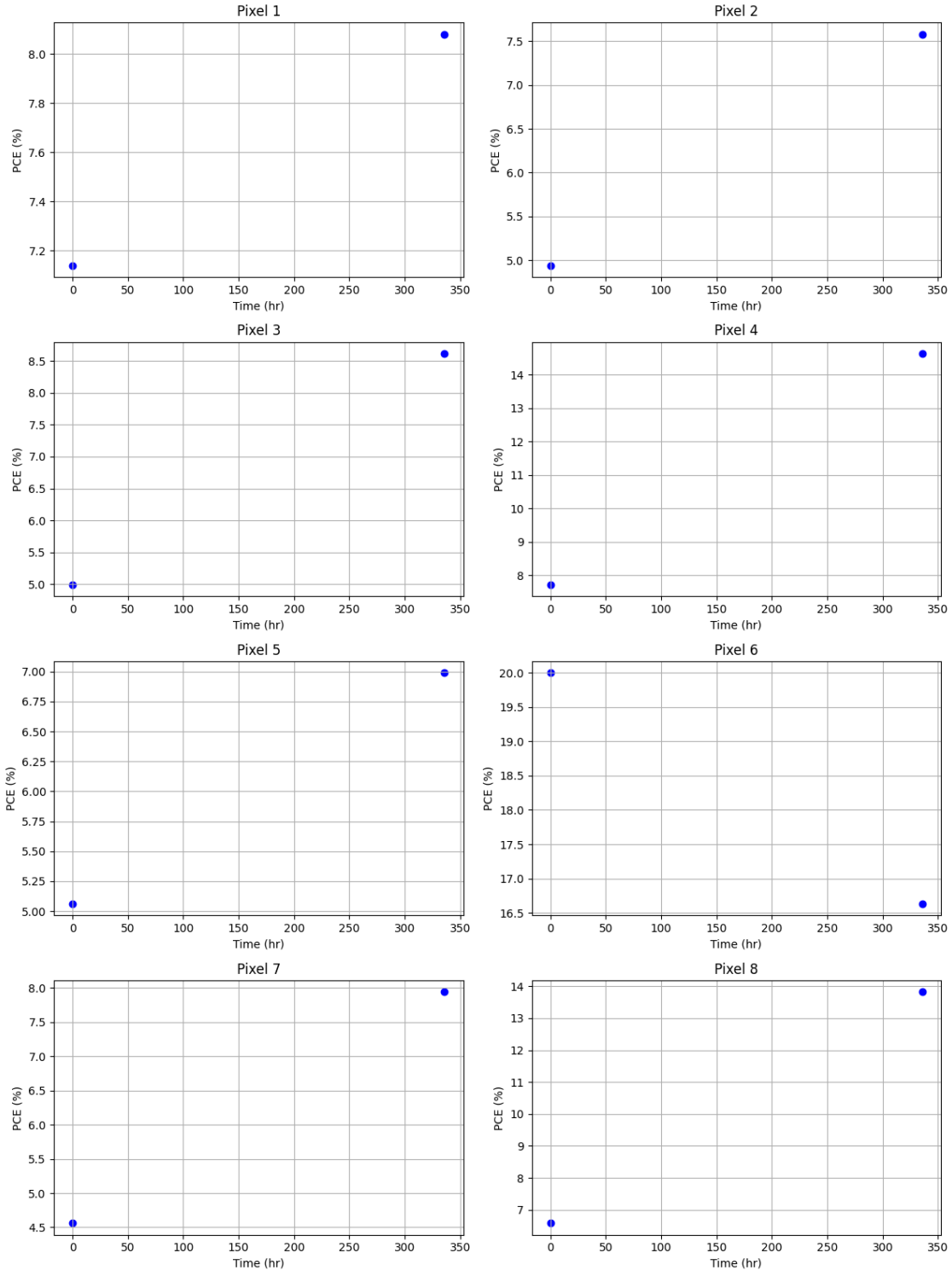
Completed - saved in CompletedColab folder and Week1 folder

5) Create figures that compare the J_{sc} measurements and the PCE measurements from the first week to the measurements from this week. Start by uploading both csv files to Colab. The ploting code has been provided below to create one figure for each parameter with multiple sub-plots. Each plot is a different pixel.

[20]:
```
# Load the data from the uploaded CSV files. You will continue to add weeks
# as you collect more data. Update file names to match yours.
week1data = pd.read_csv('/content/drive/MyDrive/Section 104 Tuesday 2:15 PM␣
 ↪Team B/CompletedColabs/WeekOne/2026_01_27_jsc_pce_cellR22.csv')
week3data = pd.read_csv('/content/drive/MyDrive/Section 104 Tuesday 2:15 PM␣
 ↪Team B/Week3/2026_02_10_jsc_pce_cellR22.csv')


# Week 1: January 27, 2026 → Time = 0 hr
# Week 3: February 10, 2026 → Time = 336 hr (14 days later)
week1data['Time (hr)'] = 0
week3data['Time (hr)'] = 336

# I reformatted the .csv file to show the pixel number without the date so i␣
 ↪had to change this line
week1data['Pixel Number'] = week1data['Pixel'].astype(int)
week3data['Pixel Number'] = week3data['Pixel'].astype(int)


# Combine the data
combined_df = pd.concat([week1data, week3data])

# Plot for Jsc (4x2 grid)
# The loop goes through each unique Pixel Number in the dataset (e.g., 1-8).
# For each pixel, it selects the relevant subset of data and plots Jsc vs. Time
# on the correct subplot. The safeguard (if idx >= len(axes)) ensures the code
# won't break if there are fewer or more than 8 pixels. Any unused subplots are␣
 ↪hidden.

# 4 rows × 2 columns = 8 subplots
fig, axes = plt.subplots(4, 2, figsize=(12, 16))

# Flatten the axes array so you can index them as axes[0] ... axes[7]
axes = axes.flatten()

for idx, pixel in enumerate(sorted(combined_df['Pixel Number'].unique())):
```

```
    if idx >= len(axes):   # safeguard in case there are fewer than 8 pixels
        break

    subset = combined_df[combined_df['Pixel Number'] == pixel]

    ax = axes[idx]
    ax.scatter(subset['Time (hr)'], subset['Jsc (mA/cm²)'], color='g')  #␣
 ↪customize style as needed

    ax.set_title(f'Pixel {pixel}')
    ax.set_xlabel('Time (hr)')
    ax.set_ylabel('Jsc (mA/cm²)')
    ax.grid(True)

# Adjust layout and show plot
plt.tight_layout()
plt.show()


# Plot for PCE (4x2 grid)
fig, axes = plt.subplots(4, 2, figsize=(12, 16))
axes = axes.flatten()

for idx, pixel in enumerate(sorted(combined_df['Pixel Number'].unique())):
    if idx >= len(axes):   # safeguard again
        break

    subset = combined_df[combined_df['Pixel Number'] == pixel]

    ax = axes[idx]
    ax.scatter(subset['Time (hr)'], subset['PCE (%)'], color='b')  # customize␣
 ↪style as needed

    ax.set_title(f'Pixel {pixel}')
    ax.set_xlabel('Time (hr)')
    ax.set_ylabel('PCE (%)')
    ax.grid(True)

plt.tight_layout()
plt.show()
```


Pixel 1
Pixel 2
Pixel 3
Pixel 4
Pixel 5
Pixel 6
Pixel 7
Pixel 8
Jsc (mA/cm²)
Time (hr)



Question 1: Discuss what you observe from looking at the comparison of the J_{sc} and PCE values from the this week and the first week. Do you have any pixels showing different behaviors after their first week of stressing?

The data from last week to recent is noticably different. It seems that most of our cells increased their Jsc by a substantial amount. Cell 6 actually decreased in Jsc but we also had some weird readings for that cell the first week. The percent increase for the PCE was much less drastic compared to Jsc. While all of the cells show quite different results from week to week, perhaps it will level out with more measurments.

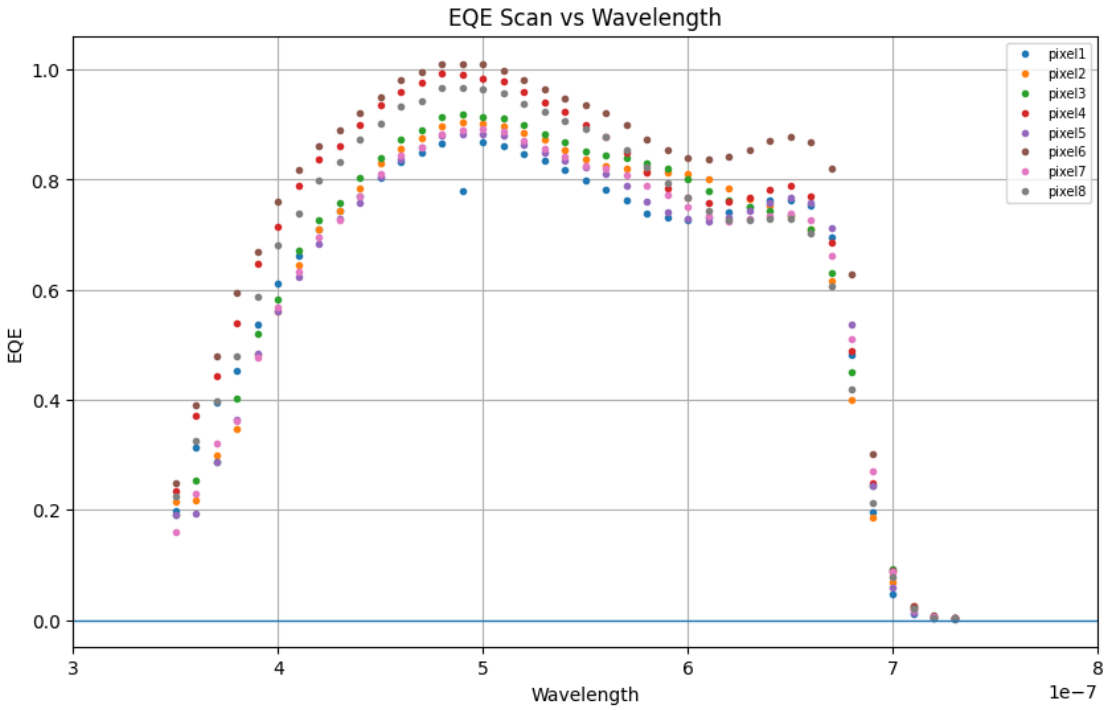
9.1 EQE Analysis

1) Calculate the EQE for each pixel you measured this week.

a) Remember you can copy and paste code from your Python tutorial or other notebooks. Consider using a loop in your python code more be more efficient.

[22]:
```
# Constants
h = 6.62607015e-34  # Planck's constant (Joule second)
c = 3.0e8  # Speed of light (meters per second)
e = 1.602176634e-19  # Elementary charge (Coulombs)
active_area_cm2 = 0.14  # Active area in cm^2
active_area_m2 = active_area_cm2 * 1e-4  # Convert cm^2 to m^2


# Read the data from the CSV files
powerCell = "/content/drive/MyDrive/Section 104 Tuesday 2:15 PM Team B/Week3/
 ↪EQE_Data/2026_02_10_power_cellR22.csv"
power_data = pd.read_csv(powerCell)

# Convert units
power_W = power_data['Power_mean (uW)'] * 1e-6       # Convert uW to W


folder = "/content/drive/MyDrive/Section 104 Tuesday 2:15 PM Team B/Week3/
 ↪EQE_Data"


files = sorted(os.listdir(folder))
eqeData = {}
i = 0

powerData = pd.read_csv(folder + "/" + "2026_02_10_power_cellR22.csv")
powerW = powerData['Power_mean (uW)'] * 1e-6       # Convert uW to W

for f in files:
  nameList = f.split("_")
  check = nameList[3]

  if check == "power":
    continue
  else:
```

```
    name = nameList[5].replace(".csv", "")
    currentData = pd.read_csv(folder + "/" + f)
    currentA = currentData['Current_mean (nA)'] * 1e-9  # Convert nA to A
    wavelengthMeters = currentData['Wavelength (nm)'] * 1e-9  # Convert nm to␣
 ↪meters

  eqe = (currentA / powerW) * (h * c / (e * wavelengthMeters))


  eqeColumns = {
      "Wavelength": wavelengthMeters,
      "EQE": eqe.round(4)
  }

  eqeData[name] = pd.DataFrame(eqeColumns);

plt.figure(figsize=(10, 6))

for name, df in eqeData.items():
    plt.plot(df['Wavelength'], df['EQE'], ".", label=name)


plt.axhline(0, linewidth=1)
plt.axvline(0, linewidth=1)
plt.xlim(3 * 1e-7, 8 * 1e-7)
plt.xlabel("Wavelength")
plt.ylabel("EQE")
plt.title("EQE Scan vs Wavelength")
plt.legend(fontsize=7)
plt.grid(True)
plt.show()
```



2) Now we will create a csv file for your EQE data to simplify plotting multiple weeks worth of data on one plot. The code below will help to create a csv file combining all your pixel EQE data from this week.

[23]:
```
# This code creates a single csv file combining your EQE results for all pixels.
# Update the variable and file names to match your data.

import pandas as pd

# You can use any pixel's current data to get the Wavelength column
# (they all share the same wavelengths)
pix1_current_data = pd.read_csv('/content/drive/MyDrive/Section 104 Tuesday 2:
 ↪15 PM Team B/Week3/EQE_Data/2026_02_10_current_cellR22_pixel8.csv')



# Create a combined DataFrame with all pixel EQE values.
# The column names should be 'eqe_pix1', 'eqe_pix2', etc.
# Make sure the variable names match what you calculated in the cell(s) above.

eqe_pix1 = eqeData['pixel1']['EQE']
eqe_pix2 = eqeData['pixel2']['EQE']
eqe_pix3 = eqeData['pixel3']['EQE']
eqe_pix4 = eqeData['pixel4']['EQE']
```

```
eqe_pix5 = eqeData['pixel5']['EQE']
eqe_pix6 = eqeData['pixel6']['EQE']
eqe_pix7 = eqeData['pixel7']['EQE']
eqe_pix8 = eqeData['pixel8']['EQE']

eqe_df = pd.DataFrame({
    'Wavelength (nm)': pix1_current_data['Wavelength (nm)'],
    'eqe_pix1': eqe_pix1,
    'eqe_pix2': eqe_pix2,
    'eqe_pix3': eqe_pix3,
    'eqe_pix4': eqe_pix4,
    'eqe_pix5': eqe_pix5,
    'eqe_pix6': eqe_pix6,
    'eqe_pix7': eqe_pix7,
    'eqe_pix8': eqe_pix8,
})

# Write to CSV - update the file name to reflect correct date and cell ID
eqe_df.to_csv('/content/drive/MyDrive/Section 104 Tuesday 2:15 PM Team B/Week3/
 ↪2026_02_10_eqe_cellR22.csv', index=False)

print("EQE data has been written to 2026_02_10_eqe_cellR22.csv")

# Download the csv files you create to your team Google Drive folder so they
# are easy to access for future weeks
```

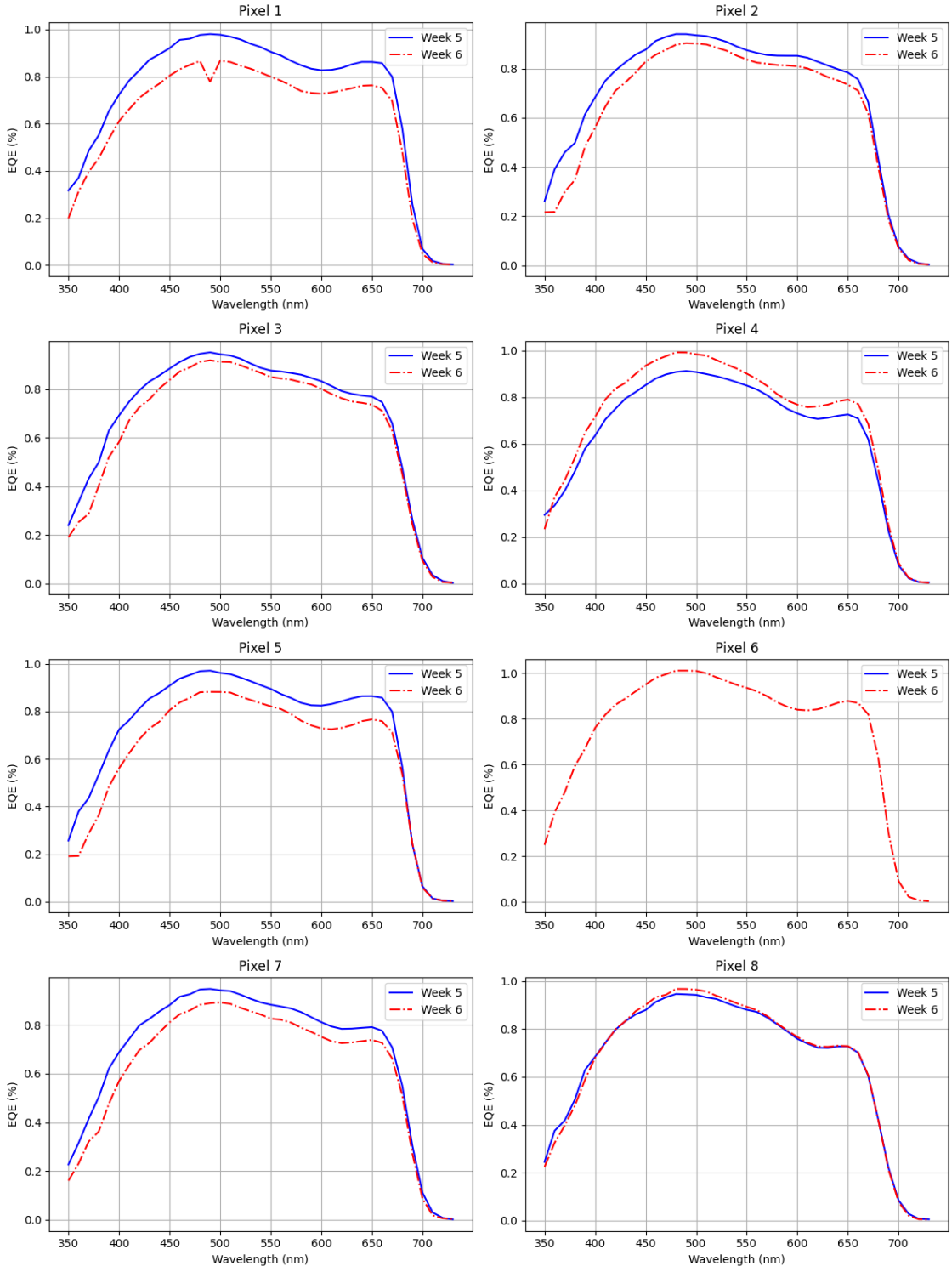
```
EQE data has been written to 2026_02_10_eqe_cellR22.csv
```

3) If you did not create a csv file of your EQE data from your first week of baseline data collection, copy and paste the code in the above cell into your Colab notebook from **the first week** to create a csv file of those EQE data as well. Make sure the varible and file names are correct. **You will need to change at least the date in csv file name.**

4) Update and use the code below to create a figure that compare the EQE measurements from the first week to the measurements from this week. The code incuded here plots data for the first pixel.

[26]:
```
# Load EQE data for each week - update file names to match yours.
# You will continue to add weeks as you collect more data.
eqe_week5 = pd.read_csv('/content/drive/MyDrive/Section 104 Tuesday 2:15 PM␣
 ↪Team B/Week2/2026_02_03_eqe_cellR22.csv')
eqe_week6 = pd.read_csv('/content/drive/MyDrive/Section 104 Tuesday 2:15 PM␣
 ↪Team B/Week3/2026_02_10_eqe_cellR22.csv')

# Create a figure with 8 subplots arranged in 4 rows x 2 columns
fig, axes = plt.subplots(4, 2, figsize=(12, 16))
axes = axes.flatten()
```

```
for i in range(8):
    ax = axes[i]
    col = f'eqe_pix{i+1}'

    if col in eqe_week5.columns:
        ax.plot(eqe_week5['Wavelength (nm)'], eqe_week5[col],
                label='Week 5', color='blue', linestyle='-')
    if col in eqe_week6.columns:
        ax.plot(eqe_week6['Wavelength (nm)'], eqe_week6[col],
                label='Week 6', color='red', linestyle='-.')

    ax.set_title(f'Pixel {i+1}')
    ax.set_xlabel('Wavelength (nm)')
    ax.set_ylabel('EQE (%)')
    ax.legend()
    ax.grid(True)

plt.tight_layout()
plt.show()

# Note: You will be doing this comparison every week. As you add more weeks,
# consider using the boilerplate code from the linked document which loads
# datasets from a list automatically for any number of weeks.
```



Question 2 Discuss what you observe from looking at the comparison of the EQE plots from this week and last week. Do you have any pixels showing different behaviors after their first week of stressing?

Yes our pixel 1 seems to have changed the most dramatically. In our EQE baseline Data, we see our pixel 1 being one of the stronger pixels. However, after stressing it seems to be one of the weaker pixels. We can also see the exact opposite change happneded to our 4th pixel. In baseline data it was lower, and in this weeks data it is higher!

10 Author Contributions (required)

1. List the name of each team member and please describe briefly how each member contributed to the work in lab this week. (e.g., taking a measurement, updating the Colab notebook, etc.)

Greg- Our coding wizard did 99% of the code, answered our AI statment, assisted in data collection

Jay- Rearanged our google drive file to help organization, assisted in data collection, Reformatted the PCE table, Answered Question 1

Madelyne- Assisted in data collection, Kept time logs, Uploaded the EQE vs. Wavelength graph, Answered Question 2.

11 Use of AI (required)

1. How your team used AI tools: Describe the specific tasks where AI-assisted tools were involved. For example, did AI help with writing, debugging, optimizing, or refining your code? Which components of the analysis did your team use AI on?
2. When your team used AI tools: Specify at what stage(s) of your programming process you used AI. Was it during initial code development, troubleshooting, etc.?
3. Which AI tools your team used: Identify the AI tools or platforms your team consulted (e.g., ChatGPT, GitHub Copilot, etc.).

AI for this lab was only used for either debugging or making the code that was given to us more compatible with our data types / names . The stages it was primarily used at were the sections where code was being given to use and we had to slightly modify. Claude was used for help where the debugging was useful.